

5.4 Concurrency on a Single File

FastAPI runs its synchronous endpoints on a thread pool, which means two requests against the same document can genuinely execute at the same time. Every mutating endpoint takes out `storage.op_lock(file_id)` before touching the file, and snapshots the pre-change state first, so undo history stays coherent even when edits are coming in quickly – as they do from the debounced autosave queue on the client.

6. API Reference

Every route below lives in `backend/main.py`. Auth works through an `httpOnly qt_token` cookie – a signed HS256 JWT that expires after 24 hours – rather than a bearer token you'd attach manually. It gets set on register and login, and cleared on logout. CORS is currently locked down to localhost origins (`http://localhost:3000`, `http://127.0.0.1:3000`, and `app://localhost`), which is exactly right for local dev and exactly wrong for anything beyond it.

6.1 Authentication

Method	Path	Description
POST	<code>/api/auth/register</code>	Create a user (name, email, password of at least 6 characters); sets the <code>qt_token</code> cookie
POST	<code>/api/auth/login</code>	Check credentials and set the <code>qt_token</code> cookie
POST	<code>/api/auth/logout</code>	Clear the <code>qt_token</code> cookie
GET	<code>/api/auth/me</code>	Return the current session's user, or <code>loggedIn: false</code>

6.2 File Lifecycle

Method	Path	Description
POST	<code>/api/files/upload</code>	Multipart upload; confirms the PDF actually opens and records its page count
GET	<code>/api/files</code>	List files (pass <code>include_deleted</code> to also see soft-deleted ones)
GET	<code>/api/files/{id}</code>	Get metadata for one file
POST	<code>/api/files/{id}/open</code>	Mark a file as opened, updating <code>opened_at</code> for recency sorting
PATCH	<code>/api/files/{id}</code>	Rename a file
GET/HEAD	<code>/api/files/{id}/content</code>	Raw PDF bytes with a no-store cache header – what the PDF.js viewer actually fetches
GET	<code>/api/files/{id}/download</code>	Raw PDF bytes with a Content-Disposition attachment header

Method	Path	Description
DELETE	/api/files/{id}	Soft delete by default; add ?permanent=true to hard-delete the file and its history
POST	/api/files/{id}/restore	Bring a soft-deleted file back

6.3 Looking Inside a Document

Method	Path	Description
GET	/api/files/{id}/outline	The PDF's bookmark/outline tree
GET	/api/files/{id}/paragraphs?page=	Editable paragraphs the text engine detected on a page
GET	/api/files/{id}/notes	List sticky-note annotations already baked into the file
PATCH	/api/files/{id}/notes/{xref}	Edit a note's text
DELETE	/api/files/{id}/notes/{xref}	Delete a note
GET	/api/files/{id}/fields	List AcroForm fields – text, checkbox, choice
POST	/api/files/{id}/fields	Bulk-set form field values by xref

6.4 Editing & History

Method	Path	Description
POST	/api/files/{id}/annotations	Bake a batch of pending edits into the PDF – see Section 4
POST	/api/files/{id}/undo	Roll back to the most recent snapshot
POST	/api/files/{id}/redo	Re-apply whatever was just undone
POST	/api/files/{id}/watermark	Stamp a text watermark – color, opacity, rotation, page range all configurable

6.5 Page Operations

Method	Path	Description
POST	/api/files/{id}/pages/rotate	Rotate selected pages by a multiple of 90°
POST	/api/files/{id}/pages/delete	Delete selected pages
POST	/api/files/{id}/pages/insert	Insert a blank page after a given index
POST	/api/files/{id}/pages/duplicate	Duplicate selected pages

Method	Path	Description
POST	/api/files/{id}/pages/reorder	Reorder pages into a new sequence
POST	/api/files/{id}/pages/extract	Pull selected pages out into a new document
POST	/api/files/{id}/split	Split into multiple documents by page-range groups
POST	/api/files/merge	Merge two or more documents into one

6.6 Export

Method	Path	Description
GET	/api/files/{id}/export?format=&pages=&dpi=	Export as PDF, PNG, or JPEG – a single image, or a ZIP if you asked for more than one page

6.7 How Errors Come Back

- Every HTTP and validation error gets normalized to the same { error, detail } JSON shape by a couple of global exception handlers.
- 404 means the file id doesn't exist; 410 Gone means the metadata is there but the actual PDF bytes on disk have gone missing somehow.
- 409 shows up for state-dependent failures – nothing left to undo/redo, or a text edit whose geometry has gone stale.
- 422 means the request was valid JSON but PyMuPDF rejected the operation itself. `bake_` annotations wraps each item in its own try/except, so one bad annotation in a batch turns into a warning rather than failing everything else in the same request.

7. Security

What follows is just an honest look at where things stand today, based on the code as it exists on sprint1. None of this is unusual for an early-stage project – it's the kind of list you'd expect to see before a first production deploy, not a sign anything went wrong.

7.1 Where Things Stand

Area	Observation
JWT signing secret	auth.py falls back to a hardcoded string if the JWT_SECRET environment variable isn't set. Any deployment that forgets to set it is signing tokens with a secret that's sitting right there in the source.
Password hashing	PBKDF2-HMAC-SHA512 at 1,000 iterations. It's implemented correctly, but 1,000 iterations is low by current standards (OWASP's current guidance for PBKDF2-SHA512 is well north of 200,000) – cheap enough that a leaked user store would be crackable offline without much effort.
Session cookie	The JWT lives in an httpOnly, SameSite=Lax cookie, which is a reasonable default against basic XSS token theft. There's no explicit Secure flag yet, and no CSRF token on state-changing requests.
CORS	Locked to localhost origins right now, which is correct for development and needs revisiting the moment this goes anywhere else.
Who can access what	File routes aren't scoped to the logged-in user – anyone who knows a valid file id can hit it. There's no ownership check in storage.py or main.py yet.
Input handling	This part's in decent shape already: filenames go through a regex allow-list (_safe_name) before touching disk, and page ranges/rotation degrees get validated before use.
Secrets in source	No .env file in the repo (correctly gitignored), but the JWT fallback value ships in source, which is really the same problem wearing a different hat.
Transport	Everything currently assumes plain HTTP on localhost – TLS isn't part of the picture yet.

7.2 What I'd Fix First

- Make JWT_SECRET required at startup – fail loudly if it's missing, instead of quietly signing with a fallback.
- Bump PBKDF2 iterations way up, or switch to Argon2id, which is built specifically for password hashing.
- Add per-user ownership on file records and check it on every /api/files/* route, not just in the UI.
- Add CSRF protection for the cookie-based auth – a double-submit token, or SameSite=Strict where the workflow allows it.
- Put TLS in front of this before it's reachable from anywhere but localhost, and turn on the Secure cookie flag.
- Rate-limit the auth endpoints – right now nothing stops repeated login attempts.

8. Testing

8.1 Where Things Stand

There isn't a test suite yet – no test files, no runner config, no CI workflow. Right now, verifying something works means running the app and trying it by hand.

8.2 What I'd Actually Test

Backend (pytest)

- The pure functions in `pdf_ops.py` first – `parse_ranges/parse_range_groups`, `hex2rgb`, the fontname mapping. Fast, no PDF needed.
- The geometry helpers in `text_engine.py` (the `_line_rect` insets, the thresholds for splitting paragraphs) against a few small hand-built PDFs.
- End-to-end tests against `bake_` annotations using fixture PDFs – specifically, proving the pass ordering actually prevents ghost text, that undo/redo round-trips get you back to byte-identical pages, and that one bad annotation in a batch produces a warning instead of a 500.
- Contract tests (FastAPI's `TestClient`) for every route in Section 6, including the 404/409/410/422 cases specifically.

Frontend (Vitest / React Testing Library + Playwright)

- Store logic first – the undo/redo transitions, the autosave debounce (`FLUSH_DELAY`, `MAX_SELECTED_DEFERS`), and `pageEpochs` actually bumping after a `bake` response comes back.
- Component-level tests for `AnnotLayer` and `PageView` – switching tools, dragging and resizing a selected annotation.
- A handful of real end-to-end flows in Playwright: upload, annotate, save, reload, and confirm it stuck; edit text, undo, confirm the original comes back; merge two files then split them and check the result.

8.3 Wiring This Into CI

A GitHub Actions workflow on every PR would go a long way – `pytest` for the backend, lint plus the Vitest suite for the frontend, and a small Playwright smoke test against a preview build. None of it exists yet, so this is really a from-scratch addition rather than an extension of something already there.

9. Deployment

9.1 Where Things Stand

Right now the only deployment artifact in the repo is `start-backends.bat` – a Windows batch file that fires up the FastAPI backend with ``uvicorn main:app --port 8001 --reload``, pointed at a hardcoded local Python path, and tells you to start the frontend separately with ``npm run dev``. There's no Dockerfile, no compose file, no pipeline of any kind.

9.2 Getting to Something Deployable

Containers

- Backend: a slim Python image running Uvicorn (behind Gunicorn, or with multiple Uvicorn workers), with the files/history/db directories mounted on a persistent volume.
- Frontend: a proper `next build`, served either through the Next.js production server or as a static export depending on how much stays dynamic.
- A docker-compose.yml tying frontend, backend, and a reverse proxy (nginx or Caddy) together, with the proxy handling TLS.

The Data Layer

- Swap db.json/users.json for SQLite as the smallest possible change, or PostgreSQL if you're planning to run more than one instance – storage.py's interface (get/create/update/list_files/soft_delete/hard_delete) is narrow enough that this shouldn't be a big rewrite.
- Move PDF files and history snapshots to object storage once there's more than one backend instance in the picture, since right now everything assumes one shared local filesystem.

Config & Environment

- Pull JWT_SECRET, allowed CORS origins, and storage paths out into environment variables, validated at startup rather than discovered at runtime.
- Add structured logging and wire a readiness probe off the existing /api/health and /health endpoints – they're already there, just not being used for anything yet.

Scaling

- The threading.Lock()-based per-file locking only works within a single process. Running more than one instance needs a real distributed lock – Postgres row locks or Redis both work – or sticky routing so the same file always lands on the same instance.

10. What's Next

10.1 Concurrency & Locking

The per-process file lock needs to become a distributed one – a Redis- or Postgres-backed lock, or a queue per file – before this can run on more than one backend instance. It's also the prerequisite for anything like real-time multi-user editing on the same document down the line.

10.2 Font Subsetting

Right now the font pool reuses whole embedded fonts when they cover what's needed. Subsetting with fonttools would mean only embedding the glyphs actually used after an edit – this matters most for documents with big CJK or symbol fonts, where editing a handful of characters shouldn't force re-embedding the entire font.

10.3 Canvas Virtualization

Pages currently render through PDF.js onto canvas elements one by one. For a long document, only mounting and rendering pages near the viewport – the store already half-supports this via viewMode/currentPage – would cut memory use and keep scrolling smooth.

10.4 Other Things Worth Doing

- Real-time collaborative editing (operational transform or CRDTs), building on the concurrency work above.
- OCR for scanned or image-only PDFs, so paragraph detection and text editing work even without a real text layer.
- Actual digital signatures – visible and cryptographic – beyond the free-form "sign" tool that's already reserved in the frontend's Tool type.
- An accessibility pass on the editor – keyboard navigation for placing annotations, ARIA labels on the canvas overlay.
- Proper API versioning (/api/v1/...) before anyone outside the project starts depending on these routes.

11. Appendix

11.1 Repository Snapshot

Item	Value
Repository	quantiradev/qt_pdf-editor
Branch reviewed	sprint1
Frontend framework	Next.js 16.2.10, React 19.2.4
Backend framework	FastAPI (≥ 0.111), PyMuPDF ($\geq 1.24.9$)
Notable commits	Initial commit, favicon/meta title, Sprint 1 auth, previewer added, editor tool fixes, editor upgrade

11.2 Glossary

- Bake – permanently applying a pending client-side edit into the actual stored PDF.
- Ghost text – the rendering glitch where old text stays visible after an edit because of a redaction/render-ordering race.
- Reflow – re-wrapping edited paragraph text inside its box using the source document's own font metrics.
- Snapshot – a full copy of a PDF's bytes taken right before a mutating operation, which is what makes undo/redo possible.